Parallel Crowd-Crush Risk Simulation Using MPI, OpenMP, and CUDA

Dwijesh P, Ayush KL, Sammit Menon

Manipal Institute of Technology, Manipal Academy of Higher Education

Abstract—We implement one deterministic, social-force-inspired pedestrian model in five execution backends: sequential CPU, OpenMP, MPI, CUDA, and MPI+OpenMP+CUDA hybrid. We compare numerical agreement, runtime scaling, parallel configuration, and simulated bottleneck indicators on a single laptop with one GPU. In a 5,000-pedestrian, 100-step case, reported median movement times were 2,841.54 ms (sequential), 938.40 ms (OpenMP), 1,436.00 ms (MPI), 138.34 ms (CUDA), and 284.19 ms (hybrid). On a separate 2,000-pedestrian configuration sweep, four OpenMP threads achieved 3.77× CPU update speedup and 0.94 parallel efficiency. In 12 s exit experiments, wider openings increased exits and reduced recorded peak density. The density and congestion labels are simulation proxies, not validated estimates of injury or real-world crowd danger.

Index Terms—pedestrian simulation, parallel computing, OpenMP, MPI, CUDA, congestion.

I. INTRODUCTION

Crowd movement near an exit combines local interactions with a global bottleneck. Social-force models represent desired motion and repulsion through continuous dynamics [1], [2]. This project asks how one shared movement rule behaves across CPU, process, GPU, and hybrid implementations, and whether its descriptive congestion measures remain consistent. The contribution is a reproducible five-backend implementation with numerical checks and local performance and exit-width studies. It is not an empirically calibrated safety predictor.

A. Problem, beneficiaries, and prior work

A single-backend simulator makes it difficult to separate model behavior from execution effects when varying crowd size or exit geometry. The practical problem is to obtain reproducible bottleneck traces while understanding the cost of several parallel execution choices. Students and researchers benefit from a shared reference implementation and comparable outputs; event planners may use the visualizations for exploratory questions only, after domain validation.

Helbing and Molnár introduced the social-force framework [1]; Helbing et al. examined escape dynamics [2]. Later empirical work showed that extreme crowd behavior challenges simple models [3]. Grandison et al. studied domain decomposition for large evacuation simulation [7]. Recent systems instead emphasize interactive density visualization [8] or calibrating agent parameters against observations [9]. Those studies motivate this work but use different models, hardware, and evaluation targets, so their runtimes cannot be compared numerically with ours. Table I summarizes the scope differences.

Work	Main focus	Relation to this work
Grandison et al. [7]	Domain decomposition	Our MPI splits IDs
Choi et al. [8]	Unity density display	Our emphasis is backend agreement
Shigenaka et al. [9]	Parameter calibration	Our model is uncalibrated
This work	Five backend comparison	One model and common outputs

TABLE I. Scope comparison with published systems; no cross-study performance ranking is implied.

The local contribution is a common state update and output contract across five backends, explicit CPU/GPU numerical tolerances, and a combined performance and congestion study on one machine. The novelty is engineering integration and controlled comparison, not a new physical crowd theory.

II. MODEL AND INDICATORS

A. Movement and exit rule

Each pedestrian has position p, velocity v, radius, preferred speed, and active status. At each step, its acceleration is $(v_{\text{desired}} - v)/\tau$ plus the sum of pedestrian, wall, and optional circular-obstacle forces divided by mass. Pair repulsion combines an exponential term in surface clearance with a linear overlap term. The direction points toward the right-wall exit. Semi-implicit Euler integration updates velocity before position, with $\Delta t = 0.02$ s and a speed cap. An agent exits only when its center crosses the opening with positive horizontal velocity and adequate radius clearance. The updater reads an immutable current state and writes a separate next state, preventing update-order dependence.

B. Congestion and risk proxy

A 1 m grid records active center count divided by each cell's actual area. A nonempty cell is congested when density is at least 2 people/m² and mean speed at most 0.5 m/s. Consecutive duration resets when that condition ends; cumulative duration sums all congested steps. The category is low with no congested cell, medium with any congested cell, and high when a currently congested cell has persisted for at least 2 s. Bottleneck occupancy and interval exit flow are also recorded. These configurable rules do not measure pressure or injury [3].

III. PARALLEL IMPLEMENTATIONS

The sequential implementation is the reference. OpenMP divides pedestrian updates over threads. MPI assigns contiguous pedestrian ID ranges to ranks; each rank holds the full current state and Allgatherv assembles the next state every step. This is ID partitioning, not spatial decomposition. CUDA launches a pedestrian-parallel kernel with configurable block size. The hybrid assigns ID ranges to MPI ranks, runs the CUDA updater for each range, and uses OpenMP plus MPI reduction for metrics. All ranks share one physical GPU, so the hybrid is a single-GPU experiment. These programming mechanisms follow [4]–[6].

Fig. 1 shows the data flow. A seeded state and the same configuration feed each backend. Each writes snapshots and summary metrics that the live interface can compare. The project writes CSV files and uses no relational database; an entity–relationship diagram would misrepresent its storage.

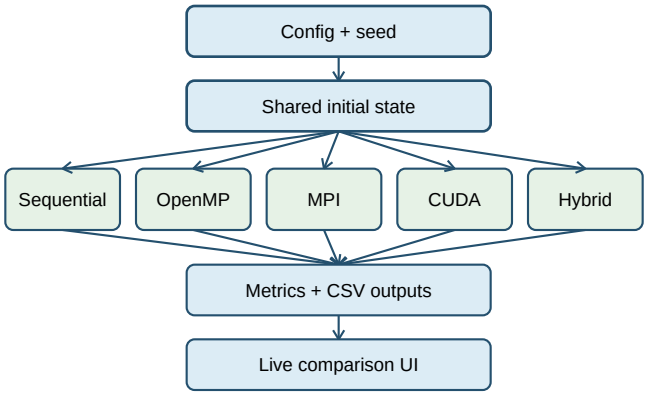

Fig. 1. Implemented component and output workflow; arrows show state or result flow. The live UI reads produced output.

IV. EXPERIMENTAL METHOD

Measurements used a Windows 11 laptop with Intel Core i7-12650H (10 physical cores, 16 logical processors), approximately 16 GiB RAM, one GeForce RTX 3050 Laptop GPU (4 GiB), CUDA Toolkit 13.1, Microsoft MPI, and MSVC. Seed 42 initialized all cases. The crowd-scaling matrix tested 500, 1,000, 2,000, and 5,000 agents for 100 steps in a 100 × 70 m room; three 100-agent, 600-step exit cases used a 20 × 12 m room and 1, 2, or 4 m openings. Each of the 35 backend/case combinations had one warmup and five measured runs (175 measured runs). OpenMP used four threads, MPI two ranks, CUDA blocks of 256, and hybrid two ranks with two OpenMP threads per rank and blocks of 256.

A second fixed 2,000-agent, 100-step sweep tested 29 configurations with one warmup and five measured runs each. Its CPU efficiency is sequential median update time divided by backend median update time and CPU worker count. CUDA and hybrid are excluded from that efficiency definition. Timed movement scopes differ: sequential includes next-buffer allocation; MPI includes communication; CUDA includes device allocation and initial upload; hybrid includes per-step transfer and communication but excludes device allocation. CSV writing and risk metric sampling are excluded from the movement benchmark. Therefore cross-backend ratios are indicative local comparisons, not equivalent end-to-end latencies.

Evaluation uses numerical agreement with the sequential state as implementation correctness, median movement time as response time, 100 steps divided by median seconds as movement-pathway throughput, and CPU update speedup/efficiency. This numerical agreement is not accuracy against observed human trajectories. Peak RAM/VRAM and user satisfaction were not instrumented, so no values are claimed for them.

V. VALIDATION AND PERFORMANCE

A. Numerical agreement

All six CTest cases passed. In the wide-exit, narrow-exit, and obstacle scenarios, parallel exit counts and active states agreed with sequential. Hybrid position and velocity values were within 0.001 absolute tolerance across 6,100 saved snapshot rows per scenario; its maximum position difference was 0.0002651 m in the narrow-exit case. Summary metric categories and counts agreed; floating values differed by at most 0.001. Near a grid boundary, small CPU/GPU rounding differences can move one pedestrian into an adjacent cell, so cell-by-cell equality is not guaranteed.

B. Crowd scaling

Fig. 2 and Table II show that the GPU setup cost dominates small cases while the GPU kernel pathway benefits from larger agent populations.

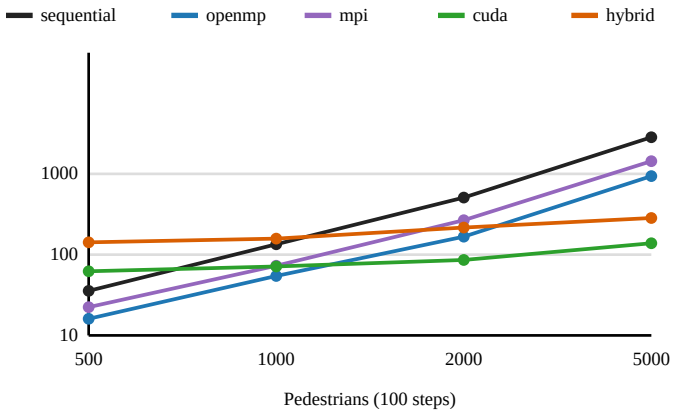

Fig. 2. Median reported movement time for the 100-step crowd scaling cases (logarithmic vertical axis); five runs per point.

Agents	Seq.	OMP	MPI	CUDA	Hybrid
500	35.6	16.1	22.4	62.2	142.0
1000	134.3	54.4	72.9	71.4	157.9
2000	508.9	166.9	266.9	85.9	216.5
5000	2841.5	938.4	1436.0	138.3	284.2

TABLE II. Median reported movement time (ms), five runs.

At 5,000 agents, the corresponding ratios to sequential are 3.03× (OpenMP), 1.98× (MPI), 20.54× (CUDA), and 10.00× (hybrid). The indicative movement throughput at that size is 35.2 steps/s sequential and 722.9 steps/s CUDA. In the 500-agent case, sequential is faster than both GPU pathways; fixed setup and communication costs are visible at small workloads. The 100-step scaling cases finished before any agent exited.

C. Configuration sweep

Table III reports CPU update speedup and efficiency relative to the sequential median; the different GPU workers do not enter this efficiency definition.

CPU path	Workers	Update ms	Speedup	Eff.
Sequential	1	491.04	1.00	1.00
OpenMP	2	246.86	1.99	0.99
OpenMP	4	130.14	3.77	0.94
OpenMP	8	146.88	3.34	0.42
MPI	2	247.27	1.99	0.99
MPI	4	150.23	3.27	0.82

TABLE III. Fixed 2,000-agent CPU sweep; medians of five runs.

OpenMP improved through four threads, whereas eight threads increased update time to 146.88 ms. CUDA block medians at 128, 256, and 512 threads were 75.10, 76.99, and 81.57 ms; their interquartile ranges overlap. For hybrid with blocks of 256 and two OpenMP threads per rank, one, two, and four ranks took 55.02, 166.84, and 342.09 ms. Sharing one GPU across ranks added transfer and communication overhead.

VI. EXIT-WIDTH AND RISK RESULTS

The risk study ran 100 and 200 pedestrians in the 20 × 12 m room for 600 steps (12 s), at exit widths 1, 2, and 4 m. It saved metric rows every 10 steps (0.2 s), while cell congestion duration accumulated every simulation step. All five backends agreed on recorded risk categories and integer counts in the 30 scenario/backend summaries; floating summary metrics differed from sequential by at most 0.001. Table IV presents the sequential values.

People	Exit m	Exited	Peak density	Congest. s	Risk
100	1	26	6	6.34	high
100	2	44	4	2.58	high
100	4	52	3	2.06	medium
200	1	46	7	6.92	high
200	2	76	6	6.10	high
200	4	97	5	5.34	high

TABLE IV. Sequential 12 s exit results; other backends agree within the stated comparison rules. Peak density is the maximum of recorded samples (people/m²); congestion is maximum cell cumulative duration.

For both crowd sizes, widening the opening increased exits and reduced recorded peak density and maximum cumulative cell congestion. At 100 agents, the 4 m case reached medium as its highest recorded category, versus high for 1 and 2 m. At 200 agents all three widths reached high under these chosen thresholds. These are observations for one seed and set of configurations, not estimates of real evacuation probability or safety.

VII. LIMITATIONS AND SECURITY

The force model omits contact friction, behavior changes, heterogeneous route choice, and empirical calibration. It uses soft pedestrian contact and a fixed population; no measured pressure or injury outcome validates the risk labels. The MPI decomposition is by ID rather than space, and the hybrid study has one GPU and one machine. Wider generality requires repeated seeds, independent hardware, measured trajectory data, and end-to-end timing with uniform boundaries. The current implementation has no measured memory profile or human-subject usability evaluation.

Configuration validation rejects unknown or duplicate keys and invalid geometry. Before exposing the interface beyond a trusted local machine, its server and file output paths would need access control and further input hardening. The risk label must be presented as a simulated indicator rather than an operational safety alert.

VIII. FUTURE WORK

Next steps are calibration and validation against measured pedestrian trajectories, repeated-seed sensitivity studies, uniform end-to-end timing, and peak CPU/GPU memory profiling. A spatial MPI partition could reduce replicated state, while multi-GPU tests would be needed before claiming distributed GPU scaling. Interface usability should be assessed with participants and a defined task before reporting satisfaction.

IX. CONCLUSION

Within this defined scope, the five backends reproduced the same exit outcomes and summary indicators. The local GPU path was fastest in the largest scaling case; four-thread OpenMP had the best measured CPU update time in the sweep. The exit-width study illustrates how the simulator can expose bottleneck trends while keeping its risk interpretation explicitly provisional.

REFERENCES

[1] D. Helbing and P. Molnár, “Social force model for pedestrian dynamics,” *Phys. Rev. E*, vol. 51, pp. 4282–4286, 1995. doi: 10.1103/PhysRevE.51.4282.

[2] D. Helbing, I. Farkas, and T. Vicsek, “Simulating dynamical features of escape panic,” *Nature*, vol. 407, pp. 487–490, 2000. doi: 10.1038/35035023.

[3] D. Helbing, A. Johansson, and H. Zein Al-Abideen, “Dynamics of crowd disasters: An empirical study,” *Phys. Rev. E*, vol. 75, 046109, 2007. doi: 10.1103/PhysRevE.75.046109.

[4] OpenMP Architecture Review Board, *OpenMP Application Programming Interface, Version 5.2*, 2021. Available: https://www.openmp.org/specifications/

[5] MPI Forum, *MPI: A Message-Passing Interface Standard, Version 4.1*, 2023. Available: https://www.mpi-forum.org/docs/mpi-4.1/

[6] NVIDIA, *CUDA Programming Guide*. Available: https://docs.nvidia.com/cuda/cuda-programming-guide/

[7] A. Grandison, Y. Cavanagh, P. J. Lawrence, and E. R. Galea, “Increasing the simulation performance of large-scale evacuations using parallel computing techniques based on domain decomposition,” *Fire Technology*, vol. 53, pp. 1399–1438, 2017. doi: 10.1007/s10694-016-0645-8.

[8] C.-H. Choi *et al.*, “Prediction and prevention of crowd-crush accidents using crowd-density simulation based on unity engine,” *Discover Applied Sciences*, vol. 7, art. 33, 2025. doi: 10.1007/s42452-024-06451-7.

[9] S. Shigenaka, S. Takami, Y. Tanigaki, S. Watanabe, and M. Onishi, “MAS-Bench: a benchmarking for parameter calibration of multi-agent crowd simulation,” *Journal of Computational Social Science*, vol. 7, pp. 2121–2145, 2024. doi: 10.1007/s42001-024-00302-6.